Lecture 2-8 — Pointer & Mouse Events

This note covers basic pointer/mouse events using the modern Pointer Events API: `pointerenter`, `pointerleave`, and `pointermove`. Pointer Events work for mouse, touch, and pen.

Code example

```
var box = document.querySelector('.box');
box.addEventListener('pointerenter', ()=>console.log('entered'));
box.addEventListener('pointerleave', ()=>console.log('left'));
box.addEventListener('pointermove', ev=>console.log(ev.clientX, ev.clientY));
```

Interactive example

X: 0, Y: 0

Output:

No events yet.

Explanation (for beginners)

- `**pointerenter**`: fires when a pointer enters the element (no bubbling).
- `**pointerleave**`: fires when a pointer leaves the element.

Powered by SPRK Technologies

- `**pointermove**`: fires while pointer moves over the element. Use throttling for heavy handlers.
- `**touch-action: none**` prevents the browser gesture (pinch/scroll) to let pointer events be delivered reliably on touch devices.

Pointer event details & tips

Pointer Events provide properties that are useful for building cross-input interactions (mouse, touch, pen):

- `**ev.pointerId**`: unique id for this pointer — useful when tracking multiple touch points.
- `**ev.pointerType**`: `'mouse' | 'touch' | 'pen'` — helps tailor UX for the input device.
- `**ev.isPrimary**`: true for the primary pointer (first finger).

Pointer capture

When you need to continue receiving pointer events even if the pointer leaves the element (common during drag), use pointer capture:

```
box.addEventListener('pointerdown', ev => {
  box.setPointerCapture(ev.pointerId);
});
box.addEventListener('pointerup', ev => {
  box.releasePointerCapture(ev.pointerId);
});
```

Throttling pointermove

Pointermove can fire many times per second (hundreds of times). That can slow your app if you do heavy work (DOM updates, calculations) on each event. Throttling limits the frequency of handling these events.

Throttle explained (simple)

Analogy: a metronome — you only perform the work on the beat. With a 50ms throttle you will at most run the handler ~20 times per second.

Powered by SPRK Technologies

Annotated throttle implementation

```
// Basic throttle implementation with timestamps
function throttle(fn, limit){
  // `last` remembers when fn last ran
  let last = 0;

  return function(...args){
    const now = performance.now();
    // if enough time passed since last call, run the function
    if(now - last >= limit){
      last = now;
      fn.apply(this, args);
    }
    // otherwise ignore the event
  };
}

// use a throttled handler for pointermove (limit to once every 50ms)
box.addEventListener('pointermove', throttle(function(ev){
  // cheap update: track coordinates
  pos.innerText = 'X: ' + ev.clientX + ', Y: ' + ev.clientY;
  // more expensive work (layout) would also be safe here at limited rate
  // doExpensiveWork(ev);
}, 50));
```

Notes on implementation choices

- The simple throttle above drops intermediate events. For some UX you may want the last event to run after the window (use a trailing call variant).
- Throttling is perfect for pointermove when you need periodic updates, not every tiny motion.

Accessibility & touch gestures

Don't disable default touch behavior unless you need to; use `touch-action` carefully and prefer pointer events over raw touch events when possible.

Powered by SPRK Technologies

Exercise

Implement a small drag box that uses pointer capture to follow the pointer while dragging.

Powered by SPRK Technologies